

ElaScout

Documentación Técnica · Athlete Intelligence Platform

Proyecto: ElaScout (MVP)

Versión: 0.1.0

Fecha: 2026-05-07

Autor: Carlos Polanco · Garaje88

Stack: Next.js 14 · Express · Firebase · Cloud Run

Repositorio: redde-elascout

Documentación Técnica del Proyecto

1. Portada

Campo	Valor
Proyecto	ElaScout — Athlete Intelligence Platform
Versión	0.1.0 (MVP)
Fecha	2026-05-07
Autor	Carlos Polanco (Garaje88)
Estado	En desarrollo — MVP funcional, módulos M1–M6 implementados, módulo de reportes con IA en pruebas
Repositorio	redde-elascout (monorepo Turborepo)
Rama principal	<code>main</code>

2. Resumen ejecutivo

ElaScout es una **plataforma SaaS de gestión, evaluación y análisis de deportistas de alto rendimiento**, con foco inicial en fútbol/soccer. Reemplaza procesos manuales (Excel, cuadernos físicos, archivos sueltos) que actualmente usan scouts, directores técnicos y preparadores físicos por una solución web centralizada que:

- Centraliza perfiles completos de deportistas (datos personales, profesionales, historial, títulos, capacidades).
- Estandariza evaluaciones físicas, técnicas y tácticas con puntajes 0–100.
- Visualiza capacidades y evolución temporal mediante gráficos radar y de líneas.
- Soporta operación **individual** (un scout autónomo) y **multi-tenant por organización** (clubes, academias) con roles y permisos.
- Genera reportes profesionales en PDF con apoyo de IA (Claude) y los envía por correo (Resend).

El stack es un monorepo TypeScript con frontend en **Next.js 14 (App Router)**, backend en **Express + Node.js**, persistencia en **Firestore**, autenticación con **Auth.js v5 (Google OAuth)** y **Firebase Admin SDK**. El despliegue es serverless: frontend en **Vercel/Cloudflare Pages**, backend en **Cloud Run**.

3. Objetivo

Objetivo principal: entregar al scout/club deportivo una plataforma digital que reemplace flujos manuales y centralice toda la información del deportista de forma estructurada, comparable y exportable.

Objetivos específicos:

1. Permitir el alta y mantenimiento de perfiles de deportistas con módulos M1–M6.
 2. Capturar evaluaciones estandarizadas (física, técnica, táctica) con puntajes en escala uniforme 0–100.
 3. Visualizar capacidades físicas con radar chart y evolución temporal con gráficos de líneas.
 4. Soportar dos modos operativos: usuario individual y organización con roles (admin, coach, scout, viewer).
 5. Generar reportes de scouting en PDF para compartir con stakeholders (DT, dirigencia, representantes).
 6. Mantener la información segura, auditable y con control de acceso por rol/organización.
-

4. Alcance

Dentro del alcance (MVP v0.1)

- Autenticación con Google (Auth.js v5) y verificación dual (Firebase ID Token + Google OIDC) en backend.
- Onboarding individual / organización con código de invitación de 6 caracteres alfanuméricos.
- CRUD completo de deportistas (módulos M1–M5).
- Sistema de evaluaciones (M6) con dos modalidades: `personal` y `game`. Caducidad automática de 90 minutos para evaluaciones `active`.
- Persistencia de scores por atleta dentro de una evaluación, con cálculo automático de promedios agregados (`physicalAvg` , `technicalAvg` , `tacticalAvg`) en el documento del atleta.
- Generación asíncrona de reportes PDF con IA (Claude Haiku) y entrega por email (Resend).
- Reglas de seguridad Firestore por rol y organización.
- Storage de fotos de atleta con límite de 5 MB por archivo.

Fuera del alcance (post-MVP)

- Dashboard avanzado con KPIs y métricas de uso (planificado v1.1).
 - Integraciones externas (TransferMarkt, Wyscout, InStat).
 - App móvil nativa.
 - Streaming/grabación de video durante evaluaciones.
 - Marketplace o módulo de transacciones entre clubes.
 - Notificaciones push o WebSockets en tiempo real.
-

5. Contexto del sistema

Actores

Actor	Descripción	Permisos típicos
Scout deportivo	Identifica talento, registra perfiles y evaluaciones	Crear atletas, evaluar
Director técnico	Consume reportes y comparativas	Crear atletas, evaluar, ver todo
Preparador físico	Registra evaluaciones físicas	Evaluar, ver todo
Representante	Visualiza fichas de sus jugadores	Lectura (post-MVP)
Admin de club	Gestiona organización, miembros y roles	CRUD completo + gestión de org
Usuario individual	Scout independiente sin organización	CRUD de sus propios atletas

Sistemas externos

- **Google OAuth 2.0** — proveedor de identidad.
- **Firebase Auth + Admin SDK** — gestión de usuarios y verificación de tokens.
- **Firebase Firestore** — base de datos NoSQL principal.
- **Firebase Storage** — almacenamiento de fotos de atletas.
- **Anthropic Claude API** — generación de contenido de reportes (modelo `claude-haiku-4-5-20251001`).
- **Resend** — entrega de correos transaccionales con adjuntos PDF.
- **Vercel / Cloudflare Pages** — hosting del frontend.
- **Google Cloud Run** — hosting del backend Express.

6. Arquitectura

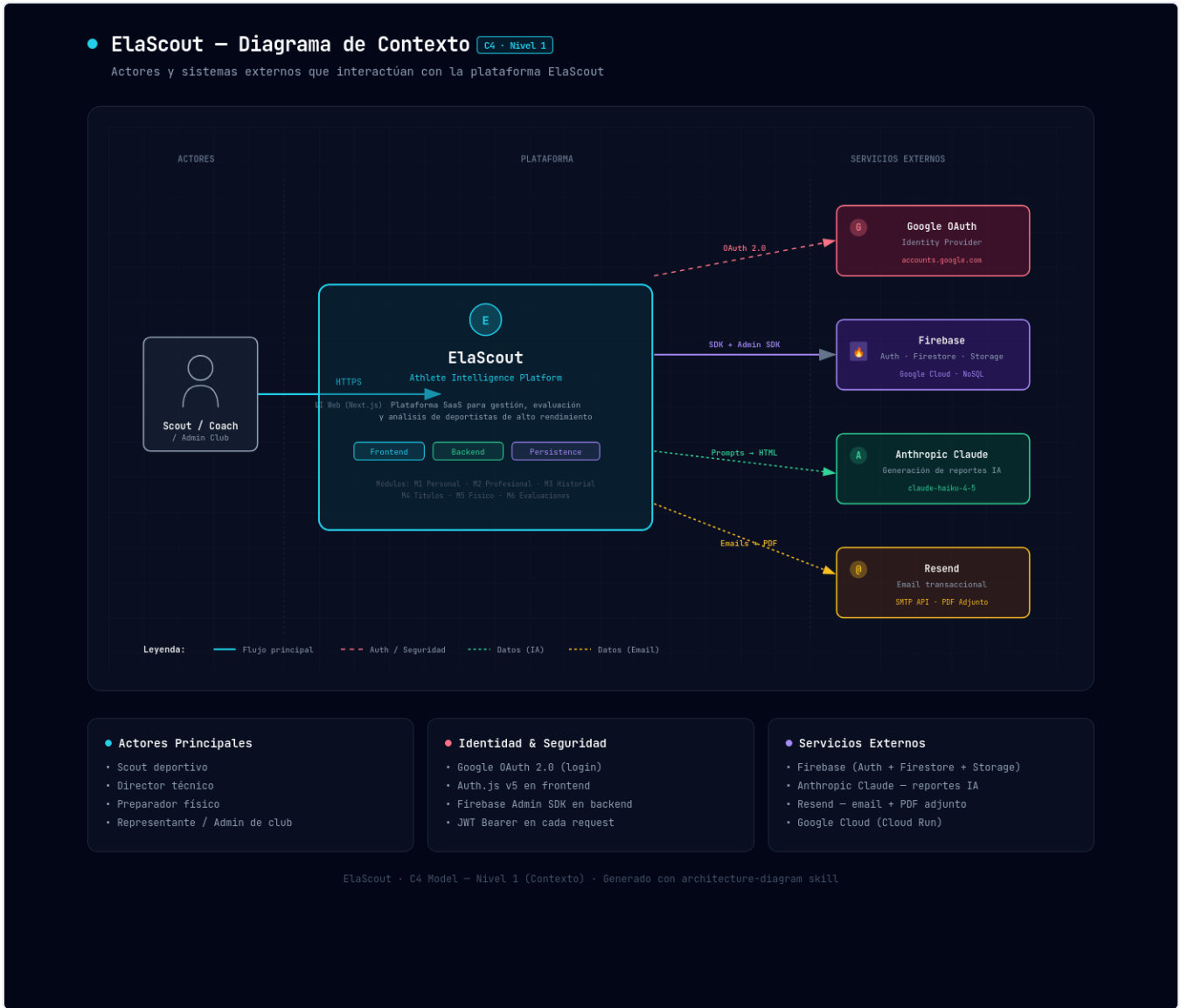
6.1 Vista general

ElaScout sigue un patrón **cliente-servidor con backend stateless** y persistencia gestionada en Firestore. La frontera de seguridad principal se aplica en dos capas redundantes:

1. **Backend (Express + Firebase Admin SDK)** — verifica el token en cada request y aplica reglas de negocio.
2. **Firestore Security Rules** — actúan como segunda capa de defensa, validando permisos en cada operación contra el documento `users/{uid}` y la subcolección `organizations/{orgId}/members/{uid}`.

El frontend usa Auth.js v5 para gestionar la sesión Google y, en paralelo, sincroniza la sesión con Firebase Auth (vía `signInWithCredential`) para que las llamadas al backend siempre lleven un Firebase ID Token verificable.

6.2 Diagrama de contexto (C4 — Nivel 1)

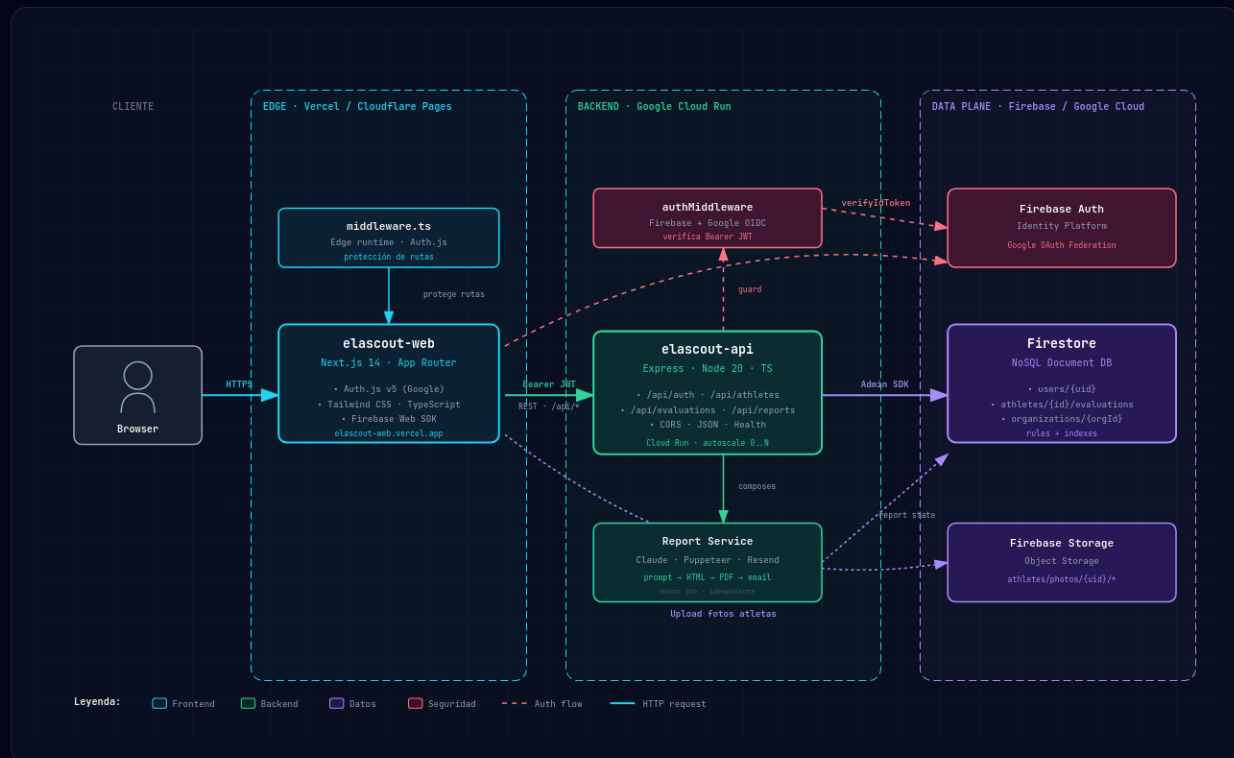


Fuente editable: docs/diagrams/01-context.drawio · docs/diagrams/01-context.spec.yaml

6.3 Diagrama de contenedores (C4 — Nivel 2)

• ElaScout – Diagrama de Contenedores C4 - Nivel 2

Frontend Next.js, Backend Express y servicios Firebase / Google Cloud



• Frontend (Edge)

- Next.js 14 App Router - TypeScript
- Auth.js v5 con Google OAuth
- Tailwind CSS - mobile-first
- Despliegue en Vercel / Cloudflare Pages

• Backend (Cloud Run)

- Express + Node 20 + TypeScript
- Patrón route - controller - service
- authMiddleware (Firebase Admin SDK)
- Report Service (Claude + Puppeteer + Resend)

• Data Plane

- Firestore: users, athletes, evaluations, orgs
- Firebase Auth: federación con Google
- Firebase Storage: fotos atletas
- Reglas de seguridad + índices compuestos

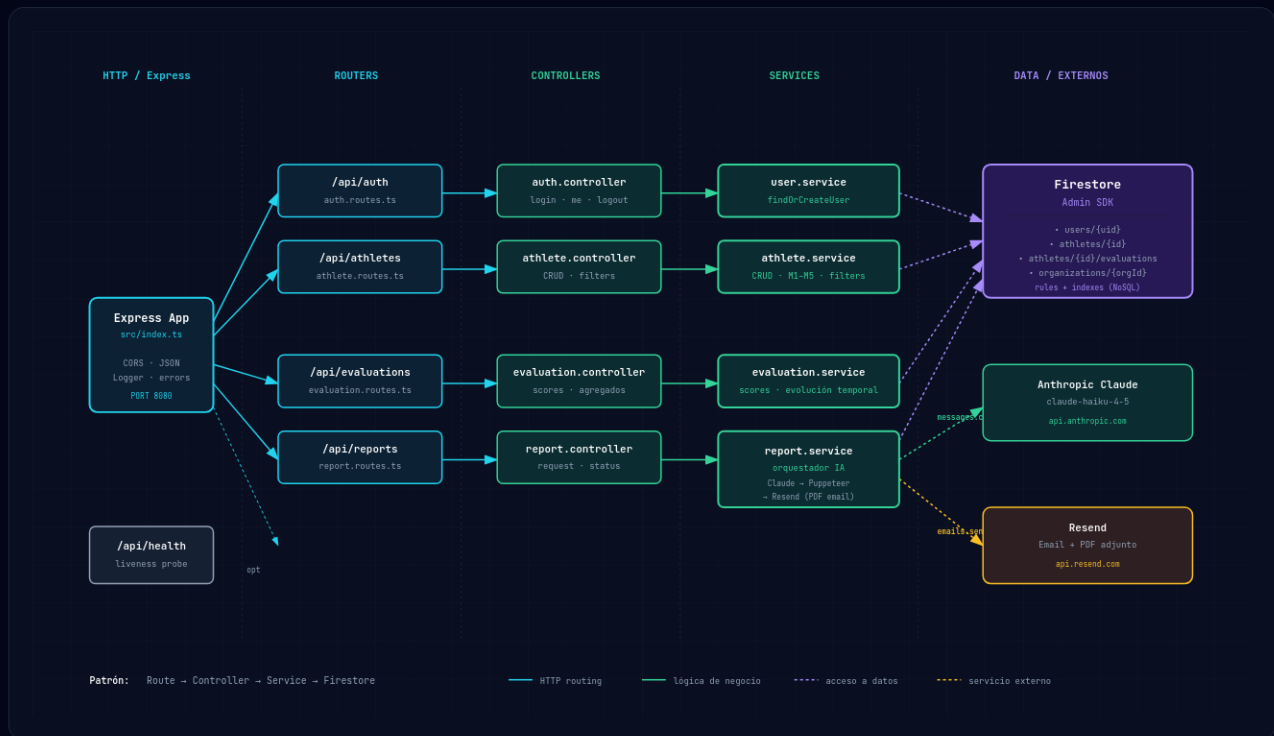
ElaScout - C4 Model - Nivel 2 (Contenedores) - Generado con architecture-diagram skill

Fuente editable: docs/diagrams/02-containers.drawio · docs/diagrams/02-containers.spec.yaml

6.4 Diagrama de componentes (backend `elascout-api`)

• ELaScout API – Diagrama de Componentes C4 - Nivel 3

Pipeline interno: Routers → Controllers → Services → Firestore + servicios externos



• Routers (Express)

- `/api/auth` - login & perfil
- `/api/athletes` - CRUD MI-M5
- `/api/evaluations` - M6 (3 tipos)
- `/api/reports` - generación IA

• Services (lógica)

- `user.service`: auto-create al primer login
- `athlete.service`: filtros + capacidades físicas
- `evaluation.service`: scores 0-100 + agregados
- `report.service`: orquesta Claude → PDF → email

• Datos & Externos

- Firestore via Firebase Admin SDK
- Anthropic Claude - Haiku 4.5
- Resend - emails transaccionales
- Puppeteer - HTML → PDF (proceso interno)

ELaScout - C4 Model - Nivel 3 (Componentes API) - Generado con `architecture-diagram skill`

Fuente editable: `docs/diagrams/03-components.drawio` · `docs/diagrams/03-components.spec.yaml`

Componentes clave del frontend (`elascout-web`):

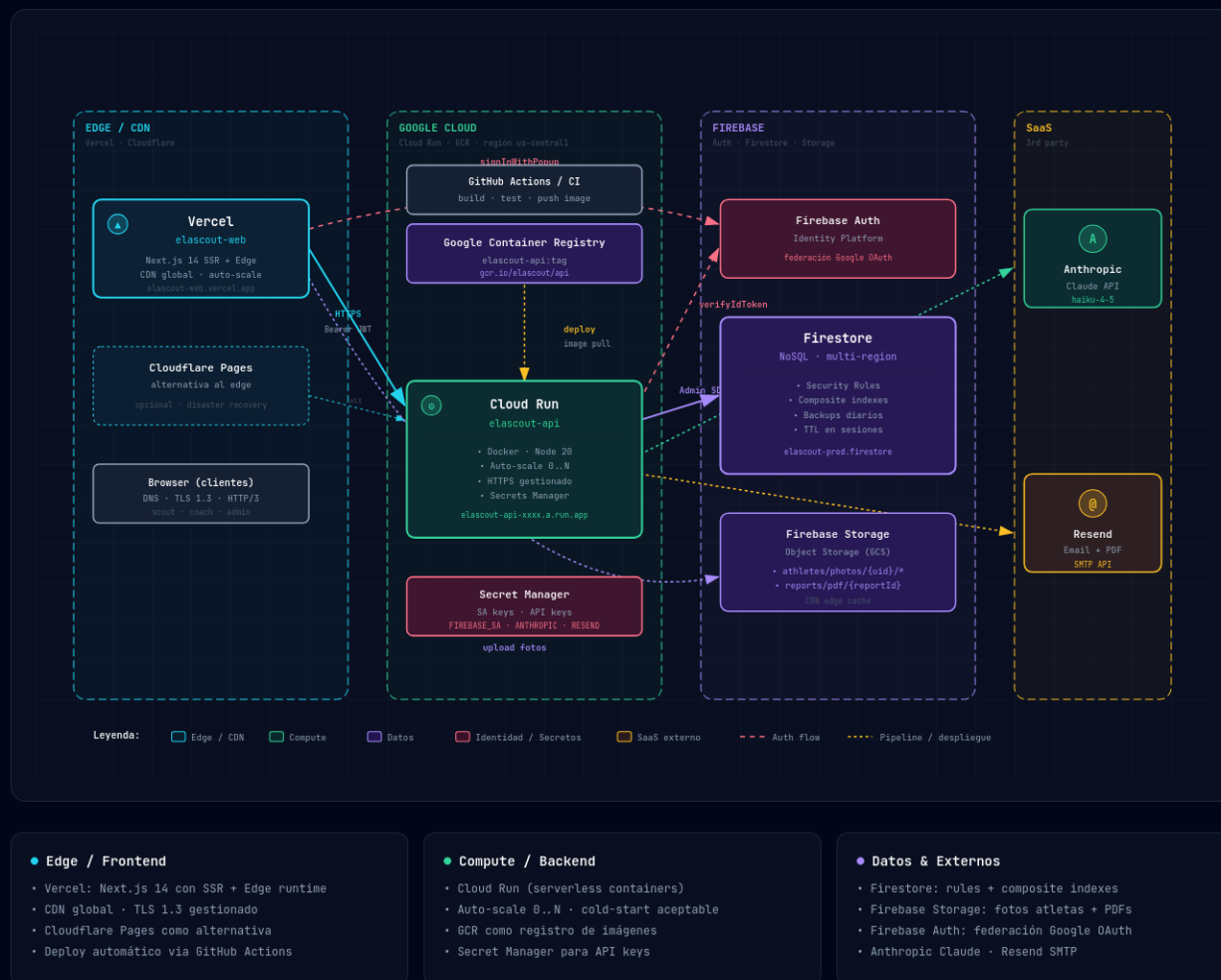
- `app/auth/signin/` — pantalla de login con Google.
- `app/onboarding/` — selección modo individual u organización.
- `app/(dashboard)/` — layout autenticado con `dashboard-client.tsx` (KPIs).
- `app/(dashboard)/athletes/` — listado, alta, edición y detalle de atletas (`athletes-search.tsx` , `athlete-form.tsx`).
- `app/(dashboard)/evaluations/` — gestión de evaluaciones (`evaluations-client.tsx`).
- `app/(dashboard)/reports/` — generación de reportes IA (`reports-client.tsx`).
- `lib/api.ts` — cliente HTTP tipado contra el backend.
- `lib/firebase-auth-provider.tsx` — provider que sincroniza sesión Auth.js con Firebase Auth.

- `auth.ts` / `auth.config.ts` — configuración Auth.js v5 con refresh de token Google.
- `middleware.ts` — protege rutas excepto `/auth/signin` y assets estáticos.

6.5 Despliegue

• ElaScout – Diagrama de Despliegue Infrastructure

Edge frontend · Cloud Run backend · Firebase data plane · Servicios externos



ElaScout · Diagrama de Despliegue · Generado con architecture-diagram skill

Fuente editable: `docs/diagrams/04-deployment.drawio` · `docs/diagrams/04-deployment.spec.yaml`

7. Stack tecnológico

7.1 Frontend

Componente	Tecnología	Versión
Framework	Next.js (App Router)	14.2.x
Lenguaje	TypeScript (strict)	5.8.x
UI	React	18.3.x
Estilos	Tailwind CSS + PostCSS + autoprefixer	3.4.x
Autenticación	Auth.js (next-auth)	5.0.0-beta.25
SDK Firebase	firebase (Web SDK)	12.10.x
Build/runtime alternativo	@cloudflare/next-on-pages + Wrangler	1.13.x / 3.95.x
Linter	ESLint + eslint-config-next	8.57.x

Convenciones:

- Componentes server por defecto; `"use client"` solo cuando se requiere estado o efectos.
- Mobile-first y responsive con breakpoints estándar de Tailwind.
- Componentes en `PascalCase`, archivos en `kebab-case`, variables y funciones en `camelCase`.
- Sin CSS Modules ni styled-components.

7.2 Backend

Componente	Tecnología	Versión
Runtime	Node.js	≥ 20
Framework HTTP	Express	4.21.x
Lenguaje	TypeScript	5.8.x
Hot reload dev	tsx	4.19.x
Firebase	firebase-admin	13.0.x
Validación tokens Google	google-auth-library	10.6.x
IA generativa	@anthropic-ai/sdk	0.80.x
HTML → PDF	puppeteer	24.40.x
Email transaccional	resend	6.9.x
Worker edge alternativo	hono + wrangler	4.6.x / 4.75.x

Patrón arquitectónico: **route** → **controller** → **service** → **Firestore**. Sin ORM (acceso directo via Firestore Admin SDK).

7.3 Base de datos

Firebase Firestore (NoSQL document database). El esquema vive en `packages/persistence/`:

- `firestore.rules` — reglas de seguridad declarativas (≈ 390 líneas).
- `firestore.indexes.json` — índices compuestos para queries frecuentes.
- `schemas/*.md` — documentación del modelo de datos.

Colecciones principales:

Colección	Documento	Subcolecciones
<code>users/{uid}</code>	Perfil de usuario	—
<code>athletes/{athleteId}</code>	Deportista	<code>evaluations/{evalId}</code> (legado), promedios agregados
<code>evaluations/{evalId}</code>	Sesión de evaluación	<code>athleteScores/{athleteId}</code>
<code>organizations/{orgId}</code>	Organización	<code>members/{uid}</code>
<code>reports/{reportId}</code>	Estado de generación de reporte	—

Índices compuestos definidos:

- `athletes`: (organizationId, createdAt) · (createdBy, createdAt) · (organizationId, nationality, createdAt) · (organizationId, position, createdAt).
- `evaluations`: (createdBy, createdAt) · (createdBy, status, createdAt) · (createdBy, type, createdAt).
- `athleteScores` (collection group): (athleteId, updatedAt).

Storage: `athletes/photos/{userId}/{...}` con límite 5 MB e `image/*` content-type.

7.4 Infraestructura

Servicio	Uso	Región
Vercel / Cloudflare Pages	Frontend Next.js (SSR + edge)	global
Google Cloud Run	Backend Express en Docker	us-central1 (configurable)
Google Container Registry	Imágenes Docker del backend	us
Firebase Firestore	Base de datos	nam5 (multi-region)
Firebase Storage	Fotos de atletas	nam5
Firebase Auth	Identidades	global
Resend	SMTP transaccional	global
Anthropic API	Generación de reportes	global

7.5 CI/CD

Estado actual:

- **Frontend (Vercel)**: push a `main` dispara deploy automático.
- **Frontend alternativo (Cloudflare Pages)**: scripts `pages:build`, `pages:preview`, `pages:deploy` en `apps/elascout-web/package.json` usando `@cloudflare/next-on-pages` + Wrangler.
- **Backend (Cloud Run)**: build manual de imagen Docker desde `apps/elascout-api/`, push a GCR y `gcloud run deploy`.
- **Cloudflare Worker (en pruebas)**: scripts `worker:dev`, `worker:deploy`, `worker:deploy:prod` con Wrangler para una variante del backend en edge runtime (`src/worker.ts`).
- **Firebase rules/indexes**: `firebase deploy --only firestore:rules,firestore:indexes`.
- **Build orquestador**: Turborepo (`turbo run build|dev|lint|test`) con caching local en `.turbo/`.

Pendiente: pipeline GitHub Actions con steps `lint` → `typecheck` → `build` → `deploy` por entorno (dev/staging/prod), tests automáticos, y separación de variables por entorno.

8. Módulos

8.1 M1 — Datos personales

Campos: `firstName`, `lastName`, `dateOfBirth`, `nationality`, `contactEmail`, `contactPhone`, `photoURL`. **Validaciones (Firestore Rules):** longitud máxima 50 chars en nombre/apellido; email validado contra regex; URL de foto debe iniciar con `http(s)://` o estar vacía.

8.2 M2 — Profesional

Campos: `position`, `secondaryPosition`, `preferredFoot` (`left` / `right` / `both`), `height` (cm, 0–300), `weight` (kg, 0–500), `currentClub`, `contractEnd`. **Posiciones válidas:** portero, defensa-central, lateral-derecho/izquierdo, centrocampista (defensivo/central/ofensivo), extremo-derecho/izquierdo, delantero-centro, segundo-delantero.

8.3 M3 — Historial deportivo

Campo: `clubHistory: ClubHistoryEntry[]` con `{ club, startYear, endYear?, position? }`. Tope 20 entradas por reglas Firestore.

8.4 M4 — Títulos y reconocimientos

Campo: `titles: TitleEntry[]` con `{ title, year, club?, category? }`. Tope 30 entradas.

8.5 M5 — Capacidades físicas

8 atributos en escala 0–100: `velocidad`, `aceleracionCorta`, `fuerzaDuelos`, `resistencia`, `potencia`, `reaccion`, `saquesLargos`, `saquesCortos`. Visualización en radar chart (frontend). Campos condicionales para porteros (`isGoalkeeper`).

8.6 M6 — Evaluaciones

Sistema de **sesiones de evaluación** desacoplado del atleta:

- **Tipos:** `personal` (un solo atleta) y `game` (múltiples atletas con formaciones de equipos).
- **Estados:** `active` (en curso, expira a los 90 minutos) → `completed` o `expired`.
- **Estructura:**
 - Documento `evaluations/{evalId}`: `type`, `title`, `notes`, `status`, `createdBy`, `startedAt`, `formation` (para tipo `game`).
 - Subcolección `evaluations/{evalId}/athleteScores/{athleteId}`: scores por categoría (`physical`, `technical`, `tactical`) más metadata `athleteId`, `evaluationId`, `updatedAt`.
- **Agregados automáticos:** al guardar scores, el servicio recalcula y persiste en el documento del atleta `physicalAvg`, `technicalAvg`, `tacticalAvg` y `evaluationCount` recorriendo todas las evaluaciones donde aparece (`updateAthleteAggregatedScores`).

- **Atributos por categoría:**

- **Físico:** velocidad, aceleración corta, fuerza en duelos, resistencia, potencia, reacción, saques largos, saques cortos (0–100).
- **Técnico:** pase, control, regate, disparo, cabeceo, presión (0–10).
- **Táctico:** posicionamiento, marcaje, desmarque, transición (0–10).

8.7 Auth y Onboarding

- Login con Google a través de Auth.js v5 (`/auth/signin`).
- `auth.ts` configura JWT strategy con refresh automático del Google `access_token` / `id_token` cuando expira; ante fallo de refresh, marca `error: "RefreshAccessTokenError"` y el provider de Firebase fuerza re-login.
- En el primer login el backend (`POST /api/auth/verify`) auto-crea el documento `users/{uid}` con `mode: null`.
- Pantalla de onboarding (`/onboarding`) permite elegir `individual` o `organization` (crear nueva o unirse con código de 6 chars).

8.8 Organizaciones y roles

- Documento `organizations/{orgId}` con `inviteCode` único de 6 caracteres `[A-Z0-9]`.
- Subcolección `members/{uid}` con `role`: `admin` | `coach` | `scout` | `viewer`.
- Reglas Firestore aplican el modelo de roles:
 - `admin`: CRUD completo + cambio de roles + eliminar miembros.
 - `coach` / `scout`: CRUD de atletas y evaluaciones.
 - `viewer`: solo lectura.

8.9 Reportes con IA (`/api/reports`)

- Endpoint `POST /api/reports` recibe `athleteIds[]`, `options`, `userName`. Responde **202 Accepted** con `reportId` y procesa de forma asíncrona.
- Pipeline:
 1. Crea registro `reports/{reportId}` con `status: processing`.
 2. Carga atletas desde Firestore.
 3. Construye prompt para Claude (`claude-haiku-4-5-20251001`) que devuelve HTML con CSS embebido y SVGs (radar charts inline, sin JS).
 4. Convierte HTML → PDF con Puppeteer (formato A4, márgenes 15/10 mm).
 5. Envía email via Resend con PDF adjunto.
 6. Actualiza `status: completed` o `failed`.
- Endpoint `GET /api/reports/:id/status` permite polling del estado por el creador (filtra por `createdBy`).

9. APIs e interfaces

Base URL: `${NEXT_PUBLIC_API_URL}` (default `http://localhost:4000/api`). Autenticación: header `Authorization: Bearer <idToken>` . El middleware acepta (1) Firebase ID Token y, como fallback, (2) Google OIDC ID Token (creando el usuario en Firebase Auth si no existe).

9.1 Auth

Método	Ruta	Body	Respuesta	Auth
POST	<code>/api/auth/verify</code>	<code>{ idToken }</code>	<code>{ user: UserDoc }</code>	No (recibe el token en body)

9.2 Athletes

Método	Ruta	Descripción	Auth
GET	<code>/api/athletes</code>	Listar con filtros <code>search</code> , <code>organizationId</code> , <code>nationality</code> , <code>position</code> , <code>ageRange</code> , <code>club</code> , <code>limit</code> (≤100), <code>startAfter</code>	✓
POST	<code>/api/athletes</code>	Crear (campos requeridos: firstName, lastName, dateOfBirth, nationality, contactEmail, contactPhone)	✓
GET	<code>/api/athletes/:id</code>	Detalle	✓
PUT	<code>/api/athletes/:id</code>	Actualizar	✓ (owner u org-mate)
DELETE	<code>/api/athletes/:id</code>	Eliminar (204)	✓ (owner o admin org)
GET	<code>/api/athletes/:id/evaluations</code>	Listar evaluaciones donde aparece (collection group query)	✓

9.3 Evaluations

Método	Ruta	Descripción	Auth
GET	/api/evaluations?status=	Listar evaluaciones del usuario	✓
POST	/api/evaluations	Crear (type ∈ personal / game , title)	✓
GET	/api/evaluations/:id	Detalle	✓
PUT	/api/evaluations/:id	Actualizar status / notes	✓
DELETE	/api/evaluations/:id	Eliminar	✓
PUT	/api/evaluations/:id/formation	Guardar formación (tipo game)	✓
GET	/api/evaluations/:id/formation	Obtener formación	✓
PUT	/api/evaluations/:id/athletes/:athleteId/scores	Guardar scores y disparar agregación	✓
GET	/api/evaluations/:id/athletes/:athleteId/scores	Leer scores	✓

9.4 Reports

Método	Ruta	Body	Respuesta
POST	/api/reports	{ athleteIds[], options, userName? }	202 { reportId, email }
GET	/api/reports/:id/status	—	{ report: { id, status, error?, createdAt } }

9.5 Códigos de error comunes

Código	Significado
400	Falta de campos requeridos o tipo inválido
401	Token ausente, inválido o expirado
403	FORBIDDEN — atleta pertenece a otra organización
404	Recurso no encontrado
500	Error inesperado; se loggea con [handlerName] ...

9.6 Health check

GET /api/health → { status: "ok", service: "elascout-api" }. Sin auth.

10. Seguridad

10.1 Autenticación

- **Frontend:** Auth.js v5 con Google OAuth (provider configurado con `prompt=select_account`, `access_type=offline` para obtener refresh token).
- **Sesión:** estrategia `jwt` (sin tabla de sesiones); tokens almacenan `accessToken`, `idToken`, `refreshToken`, `accessTokenExpires`. Refresh automático contra `https://oauth2.googleapis.com/token`.
- **Backend:** `authMiddleware` verifica Firebase ID Token (`adminAuth.verifyIdToken`); fallback a Google OIDC con `OAuth2Client.verifyIdToken`. El UID se inyecta en `req.uid`.

10.2 Autorización

Defensa en profundidad con **dos capas**:

1. **Backend services** validan ownership y pertenencia a organización antes de mutar (`athlete.service.ts` consulta `users/{uid}.organizationId` y compara contra `athlete.organizationId` antes de update/delete; lanza `FORBIDDEN`).
2. **Firestore Rules** validan en el servidor de datos:
 - `users/{uid}` : solo el dueño lee/escrbe; campo `uid` immutable; email validado por regex; `mode` y `role` validados contra enums.
 - `athletes/{id}` : lectura para autenticados; create solo si `createdBy == auth.uid` ; update por owner o `scoutOrHigher` de la org; `organizationId` solo lo cambia un `admin` ; campos `id` / `createdBy` / `createdAt` inmutables.
 - `evaluations/{id}/athleteScores/...` : tope 20 scores, valores 0–100, notas ≤ 5000 chars; el evaluador o admin de org puede actualizar.
 - `organizations/{orgId}` : read solo miembros; create con `inviteCode` formato `[A-Z0-9]{6}` ; update solo `admin` ; campos `inviteCode` / `createdBy` / `createdAt` inmutables.
 - `members/{uid}` : solo `admin` añade/cambia roles; el propio miembro puede actualizar nombre/foto y abandonar la org.

10.3 CORS

`cors({ origin: process.env.CORS_ORIGIN, credentials: true })` — sólo el frontend autorizado puede llamar al backend. Default dev: `http://localhost:3000` .

10.4 Storage

`storage.rules` restringe escritura de fotos al dueño (`request.auth.uid == userId`), con tope 5 MB y `content-type` `image/*` . Lectura permitida a cualquier autenticado.

10.5 Secretos

- Frontend: solo variables `NEXT_PUBLIC_*` y `AUTH_*` (manejadas por Vercel/Cloudflare).
- Backend: `FIREBASE_PROJECT_ID` , `FIREBASE_CLIENT_EMAIL` , `FIREBASE_PRIVATE_KEY` , `GOOGLE_CLIENT_ID` , `ANTHROPIC_API_KEY` , `RESEND_API_KEY` (Cloud Run env vars o `wrangler secret put` para Worker).
- **Service Account JSON** nunca se commitea; se inyecta como variable de entorno en Cloud Run.

10.6 Buenas prácticas pendientes

- Rate limiting (no implementado todavía a nivel API).
- Auditoría/log estructurado JSON para SIEM.
- Rotación periódica de service accounts.
- Helmet/CSP headers en Express.
- Validación de inputs con Zod (actualmente checks manuales en cada controller).

11. Escalabilidad

11.1 Capa de cómputo

- **Frontend:** Vercel/Cloudflare Pages escalan globalmente con CDN edge. SSR/SSG por ruta.
- **Backend:** Cloud Run escala de **0 a N instancias** según concurrencia. Cada instancia es **stateless**; toda la persistencia vive en Firestore.

11.2 Capa de datos

- Firestore escala automáticamente; el cuello de botella son las queries y los índices. Definidos en `firestore.indexes.json` para los patrones actuales (filtros por org/owner/posición/nacionalidad ordenados por `createdAt desc`).
- Listado de atletas con filtros aplica un patrón **fetch + filter en memoria**: trae hasta 500 docs y filtra por `search` , `nationality` , `position` , `ageRange` , `club` . Limita escalado horizontal del listado; aceptable para MVP, debe migrarse a búsqueda externa (Algolia/Meilisearch) cuando supere ~10k atletas por org.
- Agregación de promedios (`updateAthleteAggregatedScores`) recorre **todas** las `evaluations` para encontrar `athleteScores` . **Costo O(N) por escritura de score** — debe optimizarse con `collectionGroup` o un campo `athleteIds[]` en evaluations cuando crezca el dataset.

11.3 Generación de reportes

- Endpoint POST devuelve **202** y procesa en background; no bloquea el thread principal de Express.
- Bottleneck: Puppeteer requiere headless Chrome (memoria ≥ 512 MB). En Cloud Run usar instancia con suficiente RAM o externalizar a un Worker dedicado / Cloud Functions.

12. Disponibilidad

12.1 Niveles de servicio (objetivos no contratados aún)

Componente	SLO objetivo
Frontend (Vercel/CF Pages)	99.95% (depende del proveedor)
Backend (Cloud Run)	99.9% región us-central1
Firestore	99.999% multi-region nam5
Resend	99.9% (SLA proveedor)

12.2 Tolerancia a fallos

- **Cold start Cloud Run:** mitigado en producción con `min-instances=1` (recomendado, no obligatorio actualmente).
- **Refresh token fail (Auth.js):** detectado en frontend y dispara `signOut` + redirect a login.
- **Token Google stale (Firebase signInWithCredential):** captura `auth/invalid-credential` y fuerza re-login.
- **Reportes fallidos:** se persiste `status: failed` en Firestore con el mensaje de error; el cliente puede consultar via `/api/reports/:id/status`.

12.3 Backups

- Firestore: backups manuales o programados con `gcloud firestore export`. **Pendiente:** automatizar backup diario a GCS.
- Storage: versionado de objetos en bucket Firebase Storage.

13. Despliegue

13.1 Frontend (Vercel — recomendado)

1. Conectar repo `redde-elascout` en Vercel apuntando a `apps/elascout-web`.

2. Variables de entorno en Vercel:

- NEXT_PUBLIC_FIREBASE_API_KEY
- NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN
- NEXT_PUBLIC_FIREBASE_PROJECT_ID
- NEXT_PUBLIC_FIREBASE_STORAGE_BUCKET
- NEXT_PUBLIC_FIREBASE_MESSAGING_SENDER_ID
- NEXT_PUBLIC_FIREBASE_APP_ID
- NEXT_PUBLIC_API_URL
- AUTH_SECRET , AUTH_GOOGLE_ID , AUTH_GOOGLE_SECRET , AUTH_URL (URL pública del frontend)

3. Push a `main` → deploy automático.

13.2 Frontend (Cloudflare Pages — alternativo)

Desde `apps/elascout-web/`:

```
npm run pages:build      # build con @cloudflare/next-on-pages
npm run pages:preview    # preview local con wrangler
npm run pages:deploy     # publica en CF Pages (project: redde-elascout-web)
```

13.3 Backend (Cloud Run)

1. Build de imagen Docker desde `apps/elascout-api/`.
2. Push a Google Container Registry: `gcloud builds submit --tag gcr.io/<project>/elascout-api`.
3. Deploy: `gcloud run deploy elascout-api --image gcr.io/<project>/elascout-api --region us-central1 --allow-unauthenticated`.
4. Variables:
 - FIREBASE_PROJECT_ID , FIREBASE_CLIENT_EMAIL , FIREBASE_PRIVATE_KEY
 - GOOGLE_CLIENT_ID
 - CORS_ORIGIN (URL del frontend)
 - ANTHROPIC_API_KEY
 - RESEND_API_KEY , RESEND_FROM
 - PORT (Cloud Run lo inyecta como `8080`)

13.4 Cloudflare Worker (experimental)

```
cd apps/elascout-api
npm run worker:secret -- ANTHROPIC_API_KEY
npm run worker:deploy:prod
```

13.5 Firebase

```
firebase deploy --only firestore:rules
firebase deploy --only firestore:indexes
firebase deploy --only storage
```

13.6 Checklist pre-deploy

- [] `npm run lint && npm run build` exitoso desde la raíz.
- [] Variables de entorno por entorno revisadas (dev/staging/prod).
- [] Reglas Firestore actualizadas si hubo cambios de schema.
- [] Índices nuevos publicados antes de la nueva query (Firestore tarda en construirlos).
- [] Smoke test post-deploy: `/api/health`, login Google, listar atletas.

14. Observabilidad

14.1 Logs actuales

- **Backend:** `console.log/console.error` con prefijos por handler (`[listAthletes]`, `[createReport]`, `[auth]`, `[report:${id}]`). Cloud Run los envía a **Cloud Logging** automáticamente.
- **Frontend:** errores en consola de browser; Auth.js loggea su flujo en server logs de Vercel.

14.2 Métricas técnicas relevantes

- Tiempo de generación de reporte: instrumentado por etapa (`fetch`, `claude`, `pdf`, `email`, `total`) — visible en logs `[report:${id}] ...`.
- Tamaño del PDF generado.
- Conteo de atletas listados (`found %d athletes, hasMore=%s`).

14.3 Pendiente

- Métricas estructuradas con OpenTelemetry o Cloud Monitoring custom metrics.
- Tracing distribuido (frontend → backend → Firestore).
- Alertas: error rate > 1%, latencia p95 > 1s, fallos de reportes.
- Dashboard Grafana o Cloud Monitoring para SRE.

15. Decisiones arquitectónicas (ADR)

ADR-001 — Estructura monorepo (docs/decisions/001-monorepo-structure.md)

- **Estado:** Aceptada (evolucionada).
- **Contexto inicial:** monorepo simple con `src/api`, `src/web`, `src/persistence`.
- **Estado actual:** migrado a **Turborepo** con workspaces npm (`apps/*`, `packages/*`). El ADR original anticipaba esta migración.
- **Razones:** caching de builds, paralelización de tasks, deploy independiente por workspace, tipos TS compartibles.

ADR-002 — Firestore como base de datos primaria

- **Estado:** Aceptada.
- **Razones:** schema flexible para evolución del MVP, escalado automático, integración nativa con Firebase Auth, reglas de seguridad declarativas que evitan duplicar lógica de autorización.
- **Contras aceptados:** queries con joins limitadas, búsquedas full-text deben hacerse en cliente o via servicio externo, costo por documento leído.

ADR-003 — Doble capa de autenticación (Auth.js + Firebase Auth)

- **Estado:** Aceptada.
- **Razones:** Auth.js v5 permite UX moderna en Next.js (server actions, RSC) con JWT y refresh; Firebase Admin SDK simplifica verificación en backend y reglas de seguridad. Sincronizamos sesiones via `signInWithCredential(GoogleAuthProvider.credential(idToken))`.
- **Contras:** complejidad adicional en `firebase-auth-provider.tsx` y manejo de stale tokens.

ADR-004 — Reportes asíncronos con IA

- **Estado:** Aceptada.
- **Razones:** la generación con Claude + Puppeteer toma 20–60s, incompatible con HTTP request-response síncrono. Patrón **fire-and-forget**: 202 + reportId + polling.
- **Contras:** sin garantías de orden, cliente debe pollear. Considerar SSE o webhook callback en v2.

ADR-005 — Validación en doble capa (servicio + reglas Firestore)

- **Estado:** Aceptada.
- **Razones:** las reglas son la última línea de defensa (también ante clientes que llaman directo al SDK Web); los servicios validan reglas de negocio (ownership/cross-org) que serían costosas de expresar en CEL.

ADR-006 — Modelo de evaluaciones desacoplado del atleta

- **Estado:** Aceptada.

- **Contexto:** la versión inicial guardaba evaluaciones como subcolección de `athletes/{id}/evaluations/{evalId}`. La actual usa **una colección raíz** `evaluations/{evalId}` **con subcolección** `athleteScores/{athleteId}`, permitiendo evaluaciones tipo `game` con múltiples atletas.
 - **Trade-off:** se necesita un `collectionGroup` query con índice en `athleteScores.athleteId` para listar evaluaciones por atleta; los promedios se persisten en el atleta para evitar agregaciones en cada lectura.
-

16. Riesgos y limitaciones

#	Riesgo	Impacto	Probabilidad	Mitigación
R1	Búsqueda en memoria de atletas (<code>fetch 500 + filter</code>) no escala	Latencia y costos Firestore al crecer >10k docs	Media	Migrar a Algolia/Meilisearch o índice invertido propio
R2	Agregación de scores recorre toda la colección <code>evaluations</code> por escritura	Latencia y costos crecientes	Alta	Mantener lista <code>athleteIds[]</code> en cada evaluation o índice colectivo
R3	Sin tests automatizados (lint y typecheck solamente)	Regresiones silenciosas	Alta	Vitest/Jest + tests de integración con Firestore Emulator
R4	Sin rate limiting en API	Abuso por terceros con token válido	Media	<code>express-rate-limit</code> por IP/UID, alertas en Cloud Logging
R5	Puppeteer en Cloud Run: cold start largo y RAM elevada	Reportes lentos en primer uso	Media	<code>min-instances=1</code> , externalizar a un servicio dedicado
R6	Refresh token Google puede invalidarse silenciosamente	Forzar re-login al usuario	Baja	Ya gestionado en <code>auth.ts</code> / <code>firebase-auth-provider</code>
R7	Service Account de Firebase con privilegios amplios	Compromiso = acceso total al proyecto	Baja-Media	Custom roles IAM, rotación trimestral
R8	Sin backups automáticos de Firestore	Pérdida de datos por error humano o ataque	Media	Cron job <code>gcloud firestore export</code> + retención 30 días
R9	Resend en plan free tiene límites de envío	Reportes no llegan al usuario	Media	Monitorear cuota, plan de pago para producción
R10	No hay separación clara dev/ staging/prod en infra	Riesgo de pisar producción al desarrollar	Media-Alta	Tres proyectos Firebase + tres servicios Cloud Run + tres dominios

Limitaciones funcionales

- Solo fútbol/soccer en MVP; otros deportes requieren generalizar el modelo de atributos.
- Solo idioma español en la UI.
- Sin búsqueda full-text avanzada (typo-tolerant, ranking).
- Sin notificaciones (email/push) fuera del flujo de reportes.
- Sin auditoría de cambios (quién editó qué y cuándo, más allá de `updatedAt`).

17. Apéndices

A. Estructura del repositorio

```

redde-elascout/
├─ apps/
│  └─ elascout-api/                                # Backend Express
│     └─ src/
│        ├── config/firebase.ts
│        ├── controllers/                          # athlete, auth, evaluation, report
│        ├── middleware/auth.middleware.ts
│        ├── models/athlete.model.ts
│        ├── routes/                               # athlete, auth, evaluation, report
│        ├── services/                            # athlete, evaluation, report, user
│        ├── scripts/seed-athletes.ts
│        ├── lib/
│        ├── index.ts                             # Express bootstrap
│        └─ worker.ts                             # Variante Cloudflare Worker (Hono)
│     └─ CLAUDE.md
│  └─ package.json
├─ elascout-web/                                    # Frontend Next.js 14
│  └─ app/
│     ├── api/                                     # Auth.js handlers
│     ├── auth/signin/
│     ├── onboarding/
│     └─ (dashboard)/
│        ├── athletes/
│        ├── evaluations/
│        └─ reports/
│  └─ components/
│     ├── athletes/
│     ├── layout/                                 # header, sidebar, user-menu
│     └─ ui/                                     # button, input
│  └─ lib/
│     ├── api.ts                                  # Cliente HTTP tipado
│     ├── api-client.ts
│     ├── firebase.ts                             # Firebase Web SDK
│     └─ firebase-auth-provider.tsx
│  └─ auth.ts                                     # NextAuth v5 (full)
│  └─ auth.config.ts                             # NextAuth edge-safe
│  └─ middleware.ts                              # Protección de rutas
│  └─ tailwind.config.ts
│  └─ package.json
├─ packages/
│  └─ persistence/
│     ├── firestore.rules
│     ├── firestore.indexes.json
│     ├── storage.rules
│     ├── schemas/                               # athletes, evaluations, organizations, users (md)
│     └─ CLAUDE.md
├─ docs/
│  ├── architecture.md
│  ├── decisions/001-monorepo-structure.md
│  └─ runbooks/deploy.md
├─ firebase.json
├─ turbo.json
├─ package.json                                  # Workspaces npm
├─ README.md
└─ CLAUDE.md

```

B. Variables de entorno

Frontend (`apps/elascout-web/.env.local`):

```
NEXT_PUBLIC_FIREBASE_API_KEY=  
NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN=  
NEXT_PUBLIC_FIREBASE_PROJECT_ID=  
NEXT_PUBLIC_FIREBASE_STORAGE_BUCKET=  
NEXT_PUBLIC_FIREBASE_MESSAGING_SENDER_ID=  
NEXT_PUBLIC_FIREBASE_APP_ID=  
NEXT_PUBLIC_API_URL=http://localhost:4000/api  
  
AUTH_SECRET=  
AUTH_GOOGLE_ID=  
AUTH_GOOGLE_SECRET=  
AUTH_URL=http://localhost:3000
```

Backend (`apps/elascout-api/.env`):

```
PORT=4000  
CORS_ORIGIN=http://localhost:3000  
FIREBASE_PROJECT_ID=  
FIREBASE_CLIENT_EMAIL=  
FIREBASE_PRIVATE_KEY="-----BEGIN PRIVATE KEY-----\n...\n-----END PRIVATE KEY-----\n"  
GOOGLE_CLIENT_ID=  
ANTHROPIC_API_KEY=  
RESEND_API_KEY=  
RESEND_FROM="ElaScout <onboarding@resend.dev>"
```

C. Comandos de desarrollo

```
# Instalar dependencias (raíz)  
npm install  
  
# Levantar todo en paralelo  
npm run dev  
  
# Filtrar por app  
npx turbo run dev --filter=elascout-web  
npx turbo run dev --filter=elascout-api  
  
# Build / lint / test  
npm run build  
npm run lint  
npm run test  
  
# Formatear  
npm run format  
  
# Seed de datos (backend)  
cd apps/elascout-api && npm run seed
```

D. Convenciones de código

- TypeScript `strict` en todos los workspaces.
- Tailwind CSS para todo el styling (sin CSS Modules ni styled-components).
- Archivos: `kebab-case.tsx`. Componentes React: `PascalCase`. Variables/funciones: `camelCase`.
- Backend: patrón `route → controller → service → Firestore`. Sin lógica de negocio en controllers (sólo HTTP).
- Frontend: server components por defecto, `"use client"` sólo cuando necesario, mobile-first.

E. Glosario

Término	Definición
M1–M6	Módulos del perfil de deportista (ver §8)
Scout	Profesional que identifica y reporta talento deportivo
Coach	Entrenador con permisos de evaluación
Viewer	Rol de solo lectura dentro de una organización
Invite code	Código de 6 caracteres <code>[A-Z0-9]</code> para unirse a una organización
Evaluation <code>personal</code>	Sesión de evaluación de un solo atleta
Evaluation <code>game</code>	Sesión de evaluación con formaciones de dos equipos
Score agregado	Promedio (<code>physicalAvg</code> / <code>technicalAvg</code> / <code>tacticalAvg</code>) calculado y persistido en el documento del atleta
Collection group query	Query Firestore que cruza todas las subcolecciones con un mismo nombre

F. Referencias

- `README.md` — visión general del proyecto.
- `CLAUDE.md` (raíz) — guía operativa para asistentes de IA.
- `docs/architecture.md` — diagrama original del sistema.
- `docs/decisions/001-monorepo-structure.md` — ADR de monorepo.
- `docs/runbooks/deploy.md` — pasos manuales de deploy.
- `packages/persistence/schemas/*.md` — schemas detallados de cada colección.
- `packages/persistence/firestore.rules` — fuente de verdad de las reglas de seguridad.